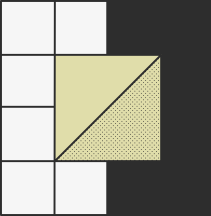


AppLab

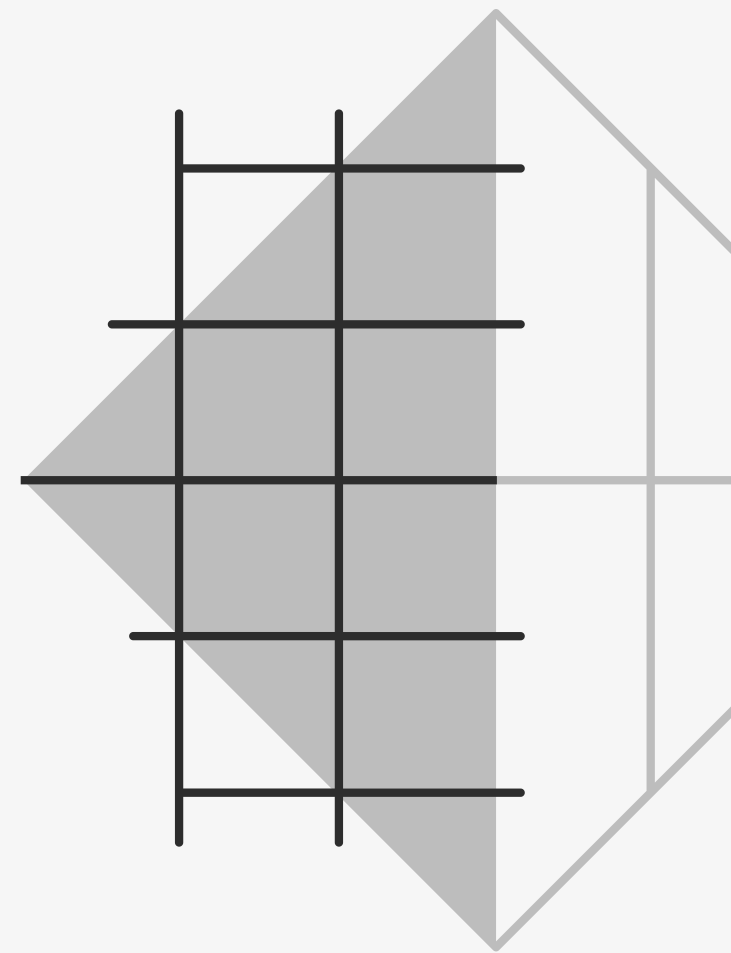
Retrieval-Augmented Generation (RAG) ChatBot

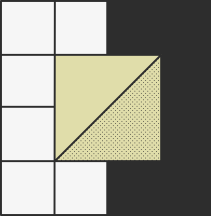




Introduction

- The RAG Chatbot Application uses Flask for the backend and HTML, CSS, and JavaScript for the frontend.
- Enables file uploads, vector embedding generation, and conversational data retrieval.





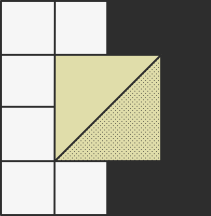
Key Features



- 
- **File Upload and Processing:** Supports .pdf, .txt, .docx; processes files into chunks; stores vector embeddings locally.
 - **Conversational Chatbot:** Provides intelligent, context-aware responses using memory.
 - **File Deletion:** Deletes uploaded files and vector stores for better storage management.
 - **Modular Design:** Separate blueprints for key functionalities.
 - **Frontend Integration:** User interface built with HTML, CSS, and JavaScript; backend connected via Fetch API.

Project Structure

```
├─ mainapp
│   ├── routes
│   │   ├── conversation_route.py
│   │   ├── uploadfile_route.py
│   │   └── deletefile_route.py
│   └── utils
│       └── file_loading_and_chunking.py
├─ __init__.py
├─ app.py
├─ requirements.txt
└─ README.md
```



Core Routes

File Upload Route

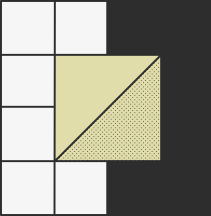
- Endpoint: upload_pdf
- Method: POST
- Handles file uploads and vector store creation

Chat Route

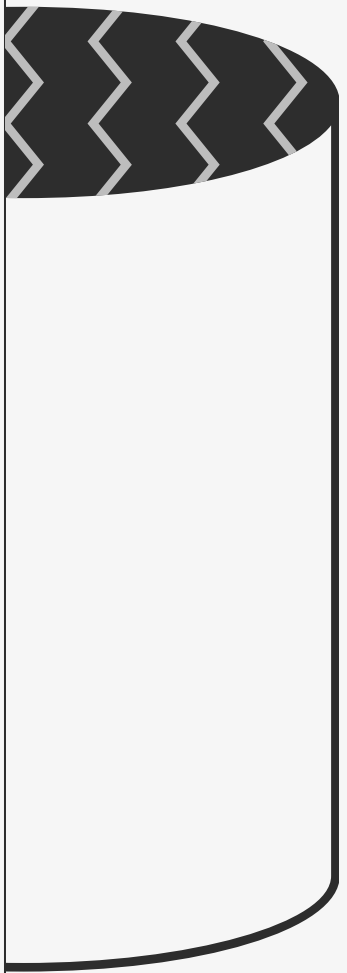
- Endpoint: /chat
- Method: POST
- Provides intelligent responses based on vector data.

Delete File Route

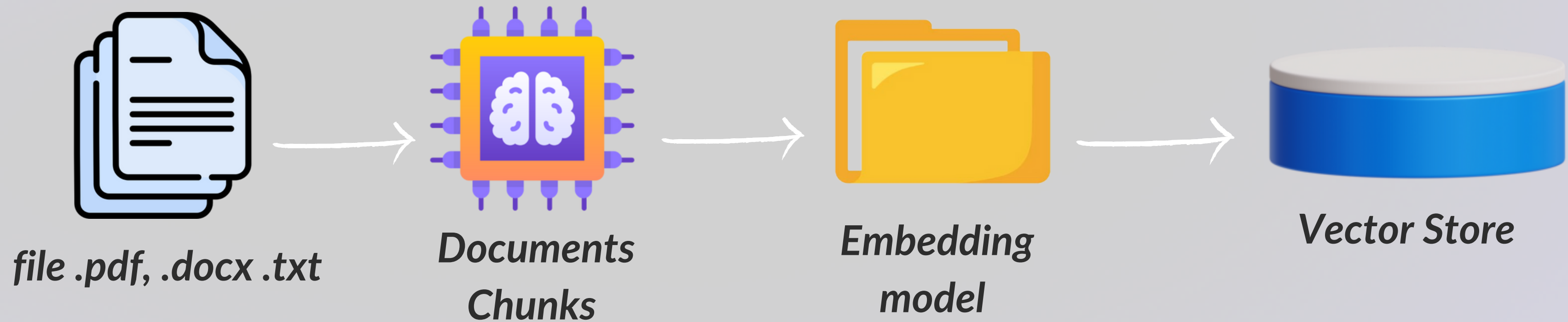
- Endpoint: /delete
- Method: DELETE
- Deletes uploaded files and vector stores.



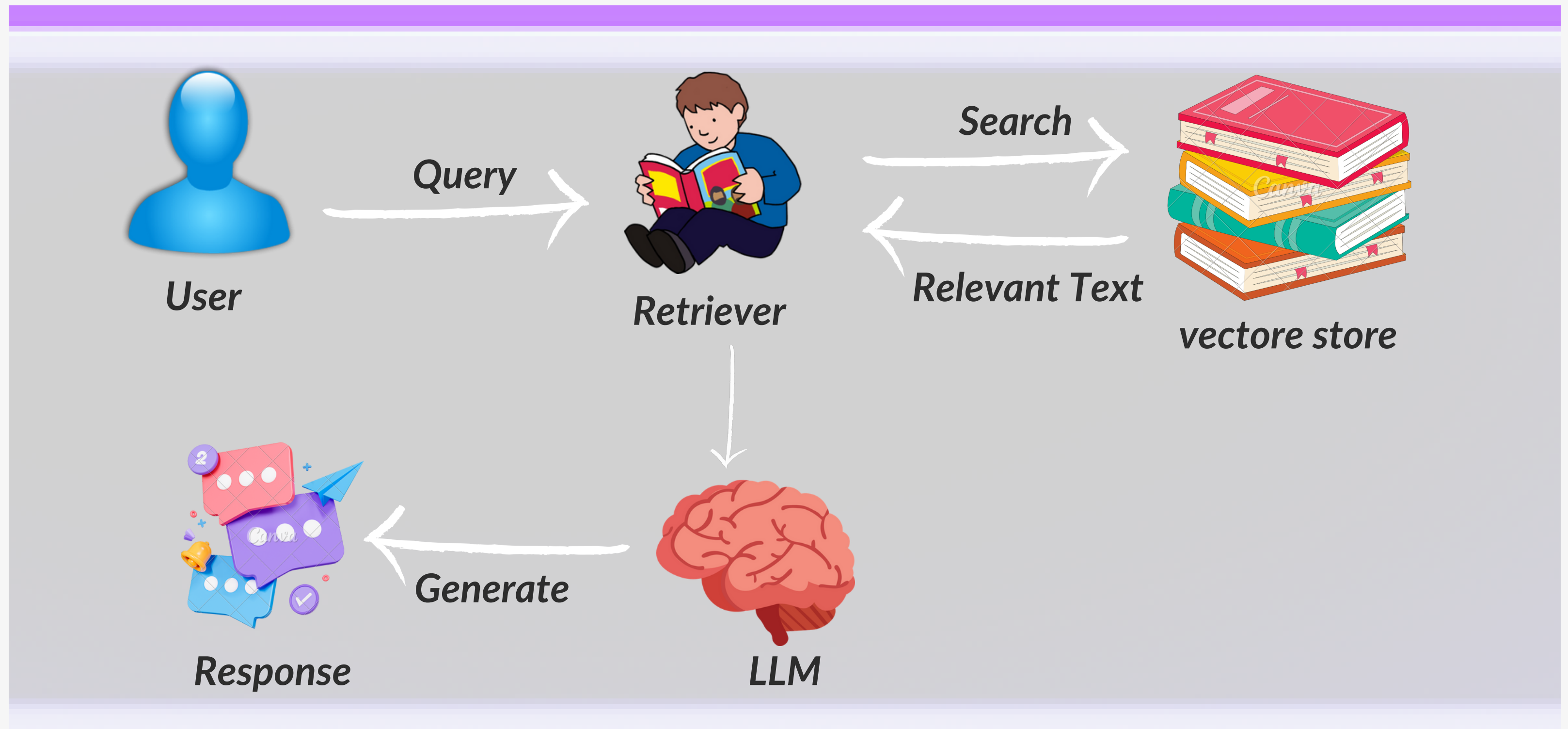
Core Functions

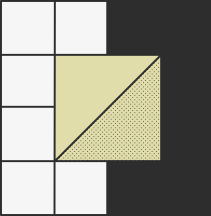
- 
- **File Processing:** Handles .pdf, .txt, .docx; creates manageable chunks.
 - **Embedding and Vector Store:** Generates and retrieves vector databases using FAISS.
 - **Chat Model:** Uses Google Generative AI with memory for context-aware interaction.

Visual Process Of file loading, splitting, embedding

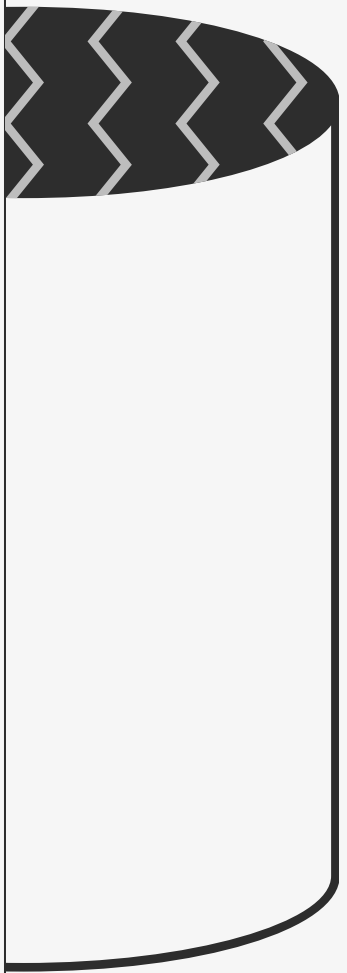


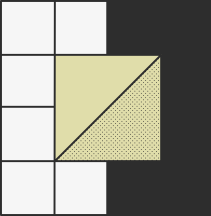
Visual Process Of Chatbot Response Generation



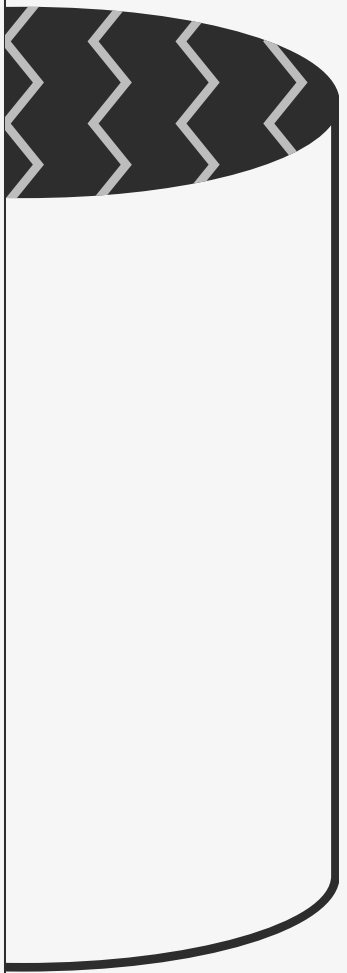


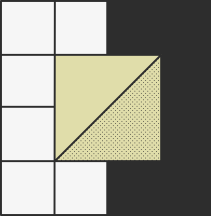
Frontend Integration

- 
- **HTML/CSS:** Provides user interface structure and styling.
 - **JavaScript Fetch API:** Enables smooth backend interaction for file uploads, chats, and deletions.

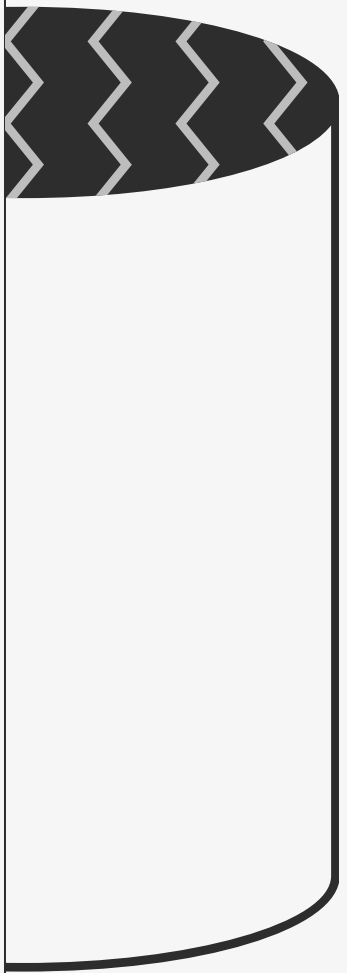


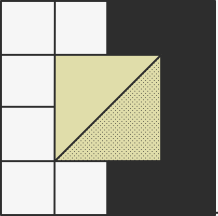
Example Workflow

- 
- **Upload File:** File is processed and vector store created.
 - **Start Conversation:** Query responses are generated based on vector data.
 - **Delete File:** Files and their vector stores are removed on request.



Future Improvements

- 
- Support for more file types.
 - Secure file uploads with authentication.
 - Enhanced UI for better interactivity.



Thank you

